📊 Systems Engineering Squad: Progress Metrics

- **Current Progress: 5%** (Commencing Week 1, Day 2 of the 3-Month Bootcamp).
- **Current Focus:** Week 1 Mandate — Memory Hygiene, RAII (Resource Acquisition Is Initialization), and Smart Pointer Architecture.

📝 The Daily Drop Blueprint

Title: Systems Engineering Squad: Week 1, Tuesday - RAII & Smart Pointer Architecture

📂 Section A: Today's Materials (The Synthesis Mandate)

You must synthesize the information from all sources below to complete today's execution mandate. Do not consult outside tutorials. *(Note: As C++ lacks a Scrimba module, you will rely entirely on enterprise documentation and industry lectures).*

1. The Video Drills (Watch These First)

- **External Anchor 1:** YouTube: The Cherno - SMART POINTERS in C++ *(The definitive guide to* unique_ptr*,* shared_ptr*, and* weak_ptr*).*
- **External Anchor 2:** YouTube: CppCon - Back to Basics: Smart Pointers *(Mandatory enterprise theory on object ownership and memory lifetimes).*
- **External Video 1:** YouTube: C++ Weekly - Stop Using new and delete *(Required viewing to understand the strict memory bans applied to your code today).*

2. Required Technical Documentation

- **Primary Architecture Doc:** LearnCpp.com - std::unique_ptr and std::make_unique *(The foundation of modern memory isolation).*
- **Specific Syntax Doc:** cppreference.com - std::shared_ptr *(Read specifically how the atomic reference counter operates).*
- **Configuration Doc:** Clang Docs: AddressSanitizer *(Required to prove your program is mathematically leak-free).*

3. Engineering Assets

- **Architecture Asset:** GitHub: Standard CMake C++ Boilerplate *(Reference for how enterprise C++ directories are structured).*
- **LLM Prompting Asset:** OpenAI Official Guide: Prompt Engineering *(Use this to ask the LLM to decode and explain complex Valgrind or AddressSanitizer stack traces).*

⚙️ Section B: The Execution Mandate (9:00 AM - 2:00 PM)

1. Execution Summary (TL;DR)

Objective: Abandon legacy C-style memory management. Build a dynamic, node-based data structure (e.g., a Singly Linked List) utilizing strict RAII principles, where memory is autonomously managed by std::unique_ptr and std::shared_ptr to mathematically guarantee zero leaks.

2. The Workflow:

- **9:00 AM - 10:00 AM (Learning):** Watch the Smart Pointer drills and CppCon lectures. Read the cppreference documentation on the overhead costs of atomic reference counting in shared pointers.
- **10:00 AM - 1:00 PM (Building):**
    - **Action 1 (Setup/Init):** Initialize your GCC/Clang environment. Configure your CMakeLists.txt to aggressively compile with the -Wall, -Wextra, -Werror, and -fsanitize=address flags.
    - **Action 2 (Core Logic):** Architect a custom LinkedList class containing a nested Node struct. You must build methods to append(), print(), and delete() nodes.
    - **Action 3 (The Memory Mandate):** The Node pointers inside your linked list must be strictly managed using std::unique_ptr. You must also implement an external class that safely observes a node using a std::shared_ptr.
- **The Non-Negotiable Rules:**
    1. **The Raw Pointer Ban:** You are explicitly forbidden from using the new and delete keywords. You may only allocate memory using std::make_unique and std::make_shared. Any pull request containing raw pointer ownership is an automatic failure.
    2. **The Leak Ban:** Your compilation process must utilize AddressSanitizer. If your terminal outputs a single byte of leaked memory, double-free, or dangling pointer upon execution, your build fails the enterprise standard.
- **1:00 PM - 1:30 PM (Hygiene):** Verify your code compiles flawlessly under the -Werror flag. Push your codebase, including the clean CMakeLists.txt, to your GitHub repository.
- **1:30 PM - 2:00 PM (LinkedIn):** Draft a technical summary of your execution. Detail the architectural differences between C++ RAII (Resource Acquisition Is Initialization) and standard Garbage Collection found in Python/Java. You must hyperlink your ProSensia admit card within the post text.

🕹️ **Section C: Submission Protocol (Due strictly by 2:00 PM)**

- **Step 1 (The Kanban Board):** Verify that this specific task card is shifted to the "Under Review" column on your team board.

- **Step 2 (The Assignments Module):** Navigate directly to today's entry in the platform's independent Assignment block. Paste both your public GitHub Repository URL and your live LinkedIn Post URL into the designated submission form fields.
- **Operational Note:** All grading and technical feedback loops operate through this portal. Do not DM Management your links.